Package 'SnapshotFDA'

October 27, 2026

Type Package

Title Covariance of a Random Function from One Observation per Curve

Version 0.1.0

Description Estimates the mean, the covariance and the score laws of a random function when each subject contributes a single noisy observation at a random time. The model has two independent, standardized, non-Gaussian scores, which identify the covariance from the law of one observation. Estimation is by sieve maximum likelihood, with the score laws in standardized two-component normal mixtures or in atomic laws on a grid. The package also predicts curves from one observation, computes identification diagnostics, and generates the data of the simulation designs of the accompanying paper.

License GPL-3

Encoding UTF-8

Depends R (>= 3.5.0)

Imports stats, utils

Contents

SnapshotFDA-package . . . 2
snapshot_align_model . . . 3
snapshot_atomic_score . . . 4
snapshot_bin_cumulants . . . 5
snapshot_cdf . . . 6
snapshot_control . . . 7
snapshot_correlation . . . 9
snapshot_covariance . . . 10
snapshot_cumulant_curves . . . 11
snapshot_data . . . 12
snapshot_density . . . 13
snapshot_design . . . 14
snapshot_empirical_cumulants . . . 15
snapshot_encode . . . 16
snapshot_fit . . . 17
snapshot_fpca . . . 19
snapshot_frequency_gram . . . 20
snapshot_loadings . . . 21

snapshot_mean . . . 22
snapshot_model . . . 23
snapshot_model_diagnostics . . . 24
snapshot_moments . . . 25
snapshot_normal_score . . . 26
snapshot_ode_coefficients . . . 27
snapshot_posterior_scores . . . 28
snapshot_quantile . . . 29
snapshot_reconstruct . . . 30
snapshot_reconstruct_quantile . . . 31
snapshot_score_cdf . . . 32
snapshot_score_cumulants . . . 33
snapshot_score_density . . . 34
snapshot_score_nd . . . 35
snapshot_simulate . . . 36
snapshot_spec . . . 37
snapshot_study_data . . . 39
snapshot_study_settings . . . 40
snapshot_trapezoid_weights . . . 41
snapshot_truth . . . 42

Index **44**

`SnapshotFDA-package` *SnapshotFDA: covariance of a random function from one observation per curve*

Description

Estimation of the mean, the covariance and the score laws of a random function from snapshot data, in which each of n independent subjects contributes one noisy observation at a random time.

Details

For t in [0, 1] the model is

$$X(t) = \mu(t) + v_1(t)\zeta_1 + v_2(t)\zeta_2, \qquad Y = X(T) + \varepsilon,$$

where the scores ζ_1, ζ_2 are independent with mean zero and variance one, $\varepsilon \sim N(0, \sigma^2)$, and T has a density on [0, 1] and is independent of the scores and the noise. The covariance is $C(s,t) = v_1(s)v_1(t) + v_2(s)v_2(t)$. The data are the pairs (T_i, Y_i), i = 1, ..., n.

With Gaussian scores the law of (T, Y) depends on the covariance only through C(t, t) + sigma^2, and time-dependent rotations of the loadings leave the snapshot law unchanged while changing C (Proposition 2.1). With non-Gaussian scores, real-analytic and linearly independent loadings, and 1, v_1^2, v_2^2 linearly independent, the law of one snapshot determines mu, sigma^2, and the loadings and score laws up to a signed permutation of the two factors, and hence C (Theorem 3.4). The cumulant curves of Y given T carry this information (Proposition 3.2, Section 3.4). Identification is weak when the scores are close to Gaussian (Theorem 4.1) or when 1, v_1^2 and v_2^2 are close to linear dependence (Proposition 4.2).

Estimation is by sieve maximum likelihood over a compact class of Chebyshev expansions and score laws (Section 5): `snapshot_spec` sets the class and the sieve, `snapshot_control` the numerical options, and `snapshot_fit` runs Algorithm 1. The restricted sieve uses standardized two-component normal mixtures for the score laws and the atomic sieve standardized laws on a grid; the estimators are consistent under the conditions of Theorem 5.6.

Fitted quantities: `snapshot_mean`, `snapshot_loadings`, `snapshot_covariance`, `snapshot_correlation`, `snapshot_moments`, `snapshot_fpca`, `snapshot_cumulant_curves`, and the conditional law of Y given T, `snapshot_density`, `snapshot_cdf`, `snapshot_quantile`. Prediction of a curve from one snapshot: `snapshot_posterior_scores`, `snapshot_reconstruct`, `snapshot_reconstruct_quantile`. Diagnostics: `snapshot_model_diagnostics`, `snapshot_ode_coefficients`, `snapshot_frequency_gram`, `snapshot_empirical_cumulants`, `snapshot_bin_cumulants`. Simulation study (Section 6): `snapshot_truth`, `snapshot_simulate`, `snapshot_design`, `snapshot_study_settings`, `snapshot_study_data`.

Author(s)

Maintainer: Ufuk Beyaztas <ufuk.beyaztas@marmara.edu.tr> (ORCID)

Authors:

- Ufuk Beyaztas <ufuk.beyaztas@marmara.edu.tr> (ORCID)
- Mina Norouzirad <m.norouzirad@fct.unl.pt> (ORCID)
- Soutir Bandyopadhyay <sbandyopadhyay@mines.edu> (ORCID)

`snapshot_align_model` *Alignment of the two factors with a reference model*

Description

Applies the signed permutation of the two factors that best matches the loadings of a reference model.

Usage

```
snapshot_align_model(object, reference, grid = seq(0, 1, length.out = 101))
```

Arguments

`object`	a `snapshot_fit` object, or a model from `snapshot_model` or `snapshot_truth`.
`reference`	a model or a `snapshot_fit` object.
`grid`	integration grid.

Details

The snapshot law determines the parameter up to the group $\mathcal{S}_2$ of the eight signed permutations S of the two factors, which act by $v \mapsto Sv$ and $\zeta \mapsto S\zeta$ (Theorem 3.4). The function chooses S to minimize the integrated squared distance of the loadings to those of the reference, and applies it to the loadings and the score laws; the law of (T, Y) and the covariance are unchanged. Comparisons of loadings and score laws with a truth, as in the metric d of Theorem 5.4, are made after this alignment.

Value

The aligned model, with the chosen permutation and signs in `alignment`.

See Also

`snapshot_loadings`, `snapshot_score_cdf`

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

# Kolmogorov distance between the fitted and the true score laws, after alignment
aligned <- snapshot_align_model(fit, d$truth, grid)
x <- seq(-6, 6, length.out = 4001)
sapply(1:2, function(k) max(abs(snapshot_score_cdf(aligned$score[[k]], x) -
                                snapshot_score_cdf(d$truth$score[[k]], x))))

## End(Not run)
```

`snapshot_atomic_score` *Score law of the atomic sieve*

Description

The standardized law $F_{J,w}$ with probabilities w on a grid of atoms.

Usage

```
snapshot_atomic_score(raw_weight, grid)
```

Arguments

`raw_weight`	nonnegative weights of the atoms.
`grid`	distinct atoms before standardization.

Details

For a probability vector w on grid points g_j, with grid mean $\bar{z}_w = \sum_j w_j g_j$ and grid variance $\varsigma_w^2 = \sum_j w_j (g_j - \bar{z}_w)^2$, the law assigns mass w_j to $(g_j - \bar{z}_w)/\varsigma_w$, so that it has mean zero and variance one. The weights are `raw_weight / sum(raw_weight)`. The atomic sieve uses the grid of the points j/J, |j| <= J^2, and requires a grid variance of at least 1/2. A law with degenerate weights is returned with `valid = FALSE`.

Value

A score law: a list with `type`, `weight`, `mean` (the standardized atoms), `variance` (zeros), the raw weights, the grid mean and variance, and `valid`.

See Also

snapshot_normal_score, snapshot_spec

Examples

```
## Not run:
# the three-point score laws of the atomic truth (Study D)
s1 <- snapshot_atomic_score(c(0.25, 0.15, 0.6), c(-1, 0, 1))
s2 <- snapshot_atomic_score(c(0.55, 0.3, 0.15), c(-1, 0, 1))
rbind(atoms = s1$mean, weights = s1$weight)

# a law on the grid of the atomic sieve with J = 2
grid <- snapshot_spec(degree = 2, sieve = "atomic")$grid
s <- snapshot_atomic_score(dbinom(0:8, 8, 0.3), grid)
snapshot_score_cumulants(s, 4)$cumulant

## End(Not run)
```

---

snapshot_bin_cumulants

Cumulants of the response within time bins under a model

Description

The first four cumulants of Y given that T falls in each bin, for T uniform within the bin.

Usage

```
snapshot_bin_cumulants(
  object,
  breaks = seq(0, 1, length.out = 11),
  nodes = NULL
)
```

Arguments

object	a snapshot_fit object, or a model from snapshot_model or snapshot_truth.
breaks	increasing bin boundaries in [0, 1].
nodes	number of Gauss-Legendre nodes in each bin.

Details

The raw moments of Y given T = t follow from the cumulant curves of snapshot_cumulant_curves; they are averaged over the bin by Gauss-Legendre quadrature and then converted to cumulants. Averaging the conditional cumulants instead would give a different quantity. The result is comparable with snapshot_empirical_cumulants.

Value

A data frame with a row for each bin: its limits, its midpoint and kappa1 to kappa4.

See Also

snapshot_empirical_cumulants

Examples

```
## Not run:
# bin cumulants of the Study D truths
breaks <- seq(0, 1, length.out = 21)
snapshot_bin_cumulants(snapshot_truth("atomic"), breaks)
snapshot_bin_cumulants(snapshot_truth("skewed"), breaks)

## End(Not run)
```

---

snapshot_cdf *Conditional distribution function of the response given the time*

Description

Distribution function of Y given T = t under a fitted or a true model.

Usage

```
snapshot_cdf(object, time, response)
```

Arguments

object	a snapshot_fit object, or a model from snapshot_model or snapshot_truth.
time	times in [0, 1].
response	responses. A time or response of length one is recycled.

Details

Given T = t, Y is a finite normal mixture with weights w_c, means $m_c(t)$ and variances $s_c^2(t)$, so $F(y \mid t) = \sum_c w_c \Phi[\{y - m_c(t)\}/s_c(t)]$; see snapshot_density. Under the true model the values $F(Y_i \mid T_i)$ are uniform on [0, 1].

Value

A vector of probabilities.

See Also

snapshot_density, snapshot_quantile

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

# probability integral transforms of the test snapshots
test <- d$test$data
u <- snapshot_cdf(fit, test$time, test$response)
hist(u, breaks = 20)

## End(Not run)
```

`snapshot_control` *Numerical options of Algorithm 1*

Description

Multistart, stopping, penalty and gradient options of `snapshot_fit`.

Usage

```
snapshot_control(
  n_start = 12,
  maxit = 800,
  outer_max = 25,
  gradient_tol = 1e-04,
  feasibility_tol = 1e-06,
  difference_step = 1e-05,
  factr = 10000,
  penalty_initial = 1,
  penalty_multiplier = 10,
  strict_feasibility = TRUE,
  trace = TRUE,
  gradient = c("central", "analytic"),
  em_iterations = 0
)
```

Arguments

`n_start`	number of random starting values.
`maxit`	maximum number of iterations of each L-BFGS-B run.
`outer_max`	maximum number of augmented Lagrangian iterations from each start.
`gradient_tol`	tolerance on the projected gradient of -L_n/n, in working and in original coordinates.
`feasibility_tol`	largest constraint violation of an accepted constrained candidate.
`difference_step`	step of the central differences.

`factr` factr of L-BFGS-B; see `optim`.

`penalty_initial` initial penalty for -L_n/n, which corresponds to c_pen = n for -L_n.

`penalty_multiplier` factor applied to the penalty.

`strict_feasibility` if `TRUE`, every computed constraint must be nonnegative at an accepted candidate. If `FALSE`, violations up to `feasibility_tol` are accepted, which can place the candidate outside the parameter class.

`trace` print the progress of the runs.

`gradient` `"central"` for the central differences of Algorithm 1, or `"analytic"` for the analytic gradient of -L_n/n, with the difference quotient where a zero-weight atomic term overflows. The penalty of the augmented Lagrangian is differenced centrally in both cases. The estimator, the boxes, the constraints and the acceptance rules do not depend on this choice.

`em_iterations` number of EM iterations that refine each starting value of the atomic sieve before the box runs.

Details

Algorithm 1 (Section 5.3) minimizes -L_n/n over the coefficient and mixture boxes by L-BFGS-B from `n_start` starting values. The mean coefficients of the starts are the least-squares coefficients of Y, the log noise variance is the log of a quarter of the residual variance, and the other coordinates are uniform on the boxes shrunk by one half. A run is accepted when the optimizer reports convergence and the projected gradient is below `gradient_tol` in every coordinate. If the best accepted run satisfies the class constraints it is returned. Otherwise each box run is continued by an augmented Lagrangian, with penalty c_pen = n for -L_n, multiplied by `penalty_multiplier` when the violation does not fall below a quarter of its previous value, for at most `outer_max` iterations, until the violation is below `feasibility_tol`. The optimizer works in box coordinates q in [-1, 1], p = center + width * q.

Value

A list of options.

See Also

`snapshot_fit`, `snapshot_study_settings`

Examples

```
## Not run:
# options of the simulation study: restricted sieve and atomic sieve
control <- snapshot_control(n_start = 12, gradient = "analytic")
atomic_control <- snapshot_control(n_start = 4, maxit = 5000, gradient = "analytic",
                                   em_iterations = 200)

## End(Not run)
```

`snapshot_correlation` *Correlation function*

Description

The correlation $\varrho(s,t) = C(s,t)/\{C(s,s)C(t,t)\}^{1/2}$ of a fitted or a true model.

Usage

```
snapshot_correlation(object, s, t = s)
```

Arguments

`object`	a `snapshot_fit` object, or a model from `snapshot_model` or `snapshot_truth`.
`s, t`	times in [0, 1].

Details

The correlation is a functional of C and is identified whenever C is; see `snapshot_covariance`. Entries with a zero variance are `NA`. The simulation study reports $\|\hat{\varrho} - \varrho\|_2$.

Value

The matrix with entries $\varrho(s_i, t_j)$.

See Also

`snapshot_covariance`

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

# L2 error of the correlation, as in the simulation study
w <- outer(snapshot_trapezoid_weights(grid), snapshot_trapezoid_weights(grid))
R <- snapshot_correlation(d$truth, grid)
R_hat <- snapshot_correlation(fit, grid)
sqrt(sum(w * (R_hat - R)^2))

## End(Not run)
```

`snapshot_covariance` *Covariance function*

Description

The covariance $C(s,t) = v_1(s)v_1(t) + v_2(s)v_2(t)$ of a fitted or a true model.

Usage

```
snapshot_covariance(object, s, t = s)
```

Arguments

`object`	a `snapshot_fit` object, or a model from `snapshot_model` or `snapshot_truth`.
`s, t`	times in [0, 1].

Details

With Gaussian scores only $C(t,t) + \sigma^2$ is determined by the snapshot law (Proposition 2.1). With non-Gaussian scores, C is identified for real-analytic loadings (Theorem 3.4) and, under further conditions, for twice continuously differentiable loadings (Theorem 3.5). On an interval, the difference of two covariances is bounded by the difference of the frequency kernels of their snapshot laws (Theorem 3.8). The error of any estimator stays bounded below with positive probability when n is small relative to the distance of the scores from Gaussian laws (Theorem 4.1) or to the distance of 1, v_1^2 and v_2^2 from linear dependence (Proposition 4.2). The simulation study reports the relative error $\|\hat{C} - C\|_2 / \|C\|_2$.

Value

The matrix with entries $C(s_i, t_j)$.

See Also

`snapshot_correlation`, `snapshot_fpca`, `snapshot_trapezoid_weights`

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

# relative covariance error, as in the simulation study
w <- outer(snapshot_trapezoid_weights(grid), snapshot_trapezoid_weights(grid))
C <- snapshot_covariance(d$truth, grid)
C_hat <- snapshot_covariance(fit, grid)
sqrt(sum(w * (C_hat - C)^2) / sum(w * C^2))
persp(grid, grid, C_hat, theta = 30, phi = 30, zlab = "C(s, t)")

## End(Not run)
```

`snapshot_cumulant_curves` *Cumulant curves of the response given the time*

Description

The cumulants $\kappa_r(t)$ of Y given T = t, r = 1, ..., max_order, or their derivatives in t.

Usage

```
snapshot_cumulant_curves(object, time, max_order = 8, derivative = 0)
```

Arguments

`object`	a `snapshot_fit` object, or a model from `snapshot_model` or `snapshot_truth`.
`time`	times in [0, 1].
`max_order`	highest order, between 2 and 20.
`derivative`	0 for the curves, 1 or 2 for their derivatives.

Details

By the independence of the scores and the noise, $\kappa_1 = \mu$, $\kappa_2 = C(t,t) + \sigma^2$ and

$$\kappa_r(t) = \kappa_r(\zeta_1) v_1(t)^r + \kappa_r(\zeta_2) v_2(t)^r, \qquad r \geq 3.$$

The snapshot law determines these curves (Lemma 3.1); for r >= 3 they satisfy the differential equation of Proposition 3.2. For the scores of Study B, $\kappa_r(\zeta_k) = \delta^r \kappa_r(Z_k)$, so the curves of order three and higher vanish as $\delta \to 0$.

Value

A matrix with a row for each time and a column for each order.

See Also

`snapshot_ode_coefficients`, `snapshot_bin_cumulants`

Examples

```
## Not run:
grid <- seq(0, 1, length.out = 101)

# third and fourth cumulant curves of the Study B truths
K_strong <- snapshot_cumulant_curves(snapshot_truth("delta", delta = 0.9), grid, 4)
K_weak <- snapshot_cumulant_curves(snapshot_truth("delta", delta = 0.35), grid, 4)
matplot(grid, cbind(K_strong[, 3:4], K_weak[, 3:4]), type = "l", lty = c(1, 1, 2, 2),
        col = c(1, 2, 1, 2), ylab = "cumulant")

## End(Not run)
```

`snapshot_data` *Snapshot data*

Description

Data frame of a snapshot sample: one observation time and one response for each of n independent subjects.

Usage

```
snapshot_data(time, response, id = seq_along(time))
```

Arguments

`time`	observation times T_i in [0, 1].
`response`	responses Y_i, one for each time.
`id`	distinct subject identifiers.

Details

The snapshot likelihood treats the rows as independent subjects, each observed once, so the identifiers must be distinct. Times must lie in [0, 1]; rescale other time ranges before fitting.

Value

A data frame with columns `id`, `time` and `response`.

See Also

`snapshot_fit`, `snapshot_simulate`

Examples

```
## Not run:
# the training sample of Study A, truth "regular", n = 2000
d <- snapshot_study_data("A", "regular", 2000)
train <- snapshot_data(d$train$time, d$train$response)
head(train)

## End(Not run)
```

snapshot_density *Conditional density of the response given the time*

Description

Density of Y given T = t under a fitted or a true model.

Usage

```
snapshot_density(object, time, response, log = FALSE)
```

Arguments

object	a snapshot_fit object, or a model from snapshot_model or snapshot_truth.
time	times in [0, 1].
response	responses. A time or response of length one is recycled.
log	return the log density.

Details

The snapshot law determines the conditional law of Y given T = t for every t (Lemma 3.1). Given T = t, Y is a finite normal mixture; on the restricted sieve

$$p_\theta(y \mid t) = \sum_{j,l=1}^{2} w_{1j} w_{2l}\, \phi\{y - \mu(t) - v_1(t) m_{1j} - v_2(t) m_{2l};\; v_1(t)^2 \varsigma_{1j}^2 + v_2(t)^2 \varsigma_{2l}^2 + \sigma^2\},$$

and on the atomic sieve the components are centred at $\mu(t) + v_1(t) z_{1j} + v_2(t) z_{2l}$ with variance σ^2. In the simulation study, the excess negative log-likelihood of test snapshots relative to the truth measures the fit of the snapshot law.

Value

A vector of densities or log densities.

See Also

snapshot_cdf, snapshot_quantile

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

# excess negative log-likelihood of the 2000 test snapshots
test <- d$test$data
mean(snapshot_density(d$truth, test$time, test$response, log = TRUE) -
     snapshot_density(fit, test$time, test$response, log = TRUE))

# density of Y given T = 0.5, fitted and true
```

```
y <- seq(-3, 4, by = 0.02)
plot(y, snapshot_density(fit, 0.5, y), type = "l", ylab = "density")
lines(y, snapshot_density(d$truth, 0.5, y), lty = 2)

## End(Not run)
```

`snapshot_design` *Cells of the simulation study*

Description

The cells of Studies A to E of Section 6.

Usage

```
snapshot_design(study = c("A", "B", "C", "D", "E"))
```

Arguments

`study`	letters of the studies, from `"A"` to `"E"`.

Details

The simulation study has five studies, each cell with 100 replications. Study A: truths `"regular"` and `"separated"`, n = 500, 1000, 2000, 5000, 10000 and 20000. Study B: truth `"delta"` with delta = 0.9, 0.7, 0.5 and 0.35, n = 2000 and 10000. Study C: truth `"beta"` with beta = 8, 2, 0.5 and 0.1, n = 2000 and 10000. Study D: truths `"atomic"` and `"skewed"`, n = 1000, 2000 and 5000, fitted on both sieves. Study E: truth `"regular"`, n = 1000, 2000 and 5000, with a paired sample of n/2 subjects observed twice for the methods that need repeated observations.

Value

A data frame with columns `study`, `scenario`, `parameter`, `n`, `replications` and `truth`, a label of the truth.

References

Beyaztas, U., Norouzirad, M. and Bandyopadhyay, S. (2026). The covariance of a random function from one observation per curve. Manuscript.

See Also

`snapshot_study_data`, `snapshot_study_settings`

Examples

```
## Not run:
design <- snapshot_design()
table(design$study)

# one cell of Study B, all replications: median and interquartile range of the
# relative covariance error
settings <- snapshot_study_settings()
```

```
w <- outer(snapshot_trapezoid_weights(settings$grid),
           snapshot_trapezoid_weights(settings$grid))
cell <- design[design$study == "B" & design$parameter == 0.5 & design$n == 2000, ]
error <- sapply(seq_len(cell$replications), function(r) {
  d <- snapshot_study_data(cell$study, cell$scenario, cell$n, cell$parameter, r)
  fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
  if (!fit$success) return(NA)
  C <- snapshot_covariance(d$truth, settings$grid)
  C_hat <- snapshot_covariance(fit, settings$grid)
  sqrt(sum(w * (C_hat - C)^2) / sum(w * C^2))
})
c(median = median(error, na.rm = TRUE), iqr = IQR(error, na.rm = TRUE))

## End(Not run)
```

`snapshot_empirical_cumulants` *Empirical cumulants of the response within time bins*

Description

The k-statistics of the responses in each time bin, with bootstrap standard errors.

Usage

```
snapshot_empirical_cumulants(
  time,
  response,
  breaks = seq(0, 1, length.out = 11),
  bootstrap = 200,
  seed = NULL
)
```

Arguments

`time`, `response`	observation times and responses.
`breaks`	increasing bin boundaries in [0, 1].
`bootstrap`	number of bootstrap samples in each bin.
`seed`	optional seed of the bootstrap. The random number state of the session is restored afterwards.

Details

The k-statistics are the unbiased estimators of the first four cumulants. In each bin they estimate the cumulants of Y given that T falls in the bin, not given one exact time; compare them with `snapshot_bin_cumulants` of a fitted model. Third and fourth cumulants away from zero show the non-Gaussian scores that identify the covariance (Lemma 3.1, Proposition 3.2). Bins with fewer than eight observations are omitted.

Value

A data frame with a row for each bin: its limits, the mean time, the number of observations, kappa1 to kappa4 and their standard errors.

See Also

snapshot_bin_cumulants, snapshot_cumulant_curves

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 5000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)

# 20 bins: empirical, fitted and true cumulants of order three
breaks <- seq(0, 1, length.out = 21)
empirical <- snapshot_empirical_cumulants(d$train$time, d$train$response, breaks,
                                          seed = 1)
fitted <- snapshot_bin_cumulants(fit, breaks)
true <- snapshot_bin_cumulants(d$truth, breaks)
plot(empirical$time, empirical$kappa3, ylim = range(empirical$kappa3, true$kappa3),
     xlab = "t", ylab = "kappa3")
lines(fitted$time, fitted$kappa3)
lines(true$time, true$kappa3, lty = 2)

## End(Not run)
```

---

snapshot_encode *Working coordinates of a model*

Description

The box coordinates q of a model in a sieve, for use as a starting value of snapshot_fit.

Usage

```
snapshot_encode(model, spec = NULL)
```

Arguments

model	a snapshot_fit object or a model.
spec	a snapshot_spec. By default the one stored in the model.

Details

Algorithm 1 works with $q \in [-1, 1]^p$, where the parameter is p = center + width * q and the widths are the coefficient and mixture boxes of snapshot_spec. Coefficients above the degrees of the sieve must be zero. Entries outside [-1, 1] show that the model lies outside the box. Starting the algorithm at the true parameter, as below, checks whether the random starts reach the largest likelihood.

Value

A named vector.

See Also

snapshot_fit

Examples

```
## Not run:
# Study B, delta = 0.35: random starts against a start at the truth
settings <- snapshot_study_settings()
d <- snapshot_study_data("B", "delta", 2000, parameter = 0.35)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
q0 <- snapshot_encode(d$truth, settings$spec)
fit0 <- snapshot_fit(d$train, settings$spec, settings$control, starts = list(q0))
c(random_starts = fit$loglik, truth_start = fit0$loglik)

## End(Not run)
```

`snapshot_fit` *Sieve maximum likelihood from snapshot data*

Description

Fits the snapshot model by Algorithm 1 over the restricted or the atomic sieve.

Usage

```
snapshot_fit(
  data,
  spec = snapshot_spec(),
  control = snapshot_control(),
  seed = NULL,
  starts = NULL,
  warm = NULL
)

## S3 method for class snapshot_fit
print(x, ...)
```

Arguments

`data`	data frame with columns `time` and `response`, and optionally distinct subject identifiers `id`; see `snapshot_data`.
`spec`	parameter class and sieve, from `snapshot_spec`.
`control`	numerical options, from `snapshot_control`.
`seed`	optional seed of the random starting values. The random number state of the session is restored afterwards.
`starts`	optional list of starting values in working coordinates, for instance from `snapshot_encode`.
`warm`	optional fit of the same sieve, for instance of lower degree, whose parameter, padded with zeros, is added as a further start.
`x`	a `snapshot_fit` object.
`...`	not used.

Details

The model is

$$X(t) = \mu(t) + v_1(t)\zeta_1 + v_2(t)\zeta_2, \qquad Y = X(T) + \varepsilon,$$

with independent standardized scores, $\varepsilon \sim N(0, \sigma^2)$ and T independent of the scores and the noise. Since the density of T does not depend on the parameter, the log-likelihood is $L_n(\theta) = \sum_{i=1}^{n} \log p_\theta(Y_i \mid T_i)$. On the restricted sieve

$$p_\theta(y \mid t) = \sum_{j,l=1}^{2} w_{1j} w_{2l}\, \phi\{y - \mu(t) - v_1(t) m_{1j} - v_2(t) m_{2l};\; v_1(t)^2 \varsigma_{1j}^2 + v_2(t)^2 \varsigma_{2l}^2 + \sigma^2\},$$

where w_{kj}, m_{kj} and ς_{kj}^2 are the weights, means and variances of the two components of the k-th score law. On the atomic sieve the score laws are atoms z_{kj} with weights w_{kj}, and the component variance is σ^2. The estimator is any point of the sieve with $n^{-1} L_n(\hat\theta_n) \geq \sup n^{-1} L_n - \eta_n$, and $\hat C_n = C_{\hat\theta_n}$ has rank at most two. By Theorem 5.6, the restricted sieve gives a consistent estimator when it contains the truth, and the atomic sieve does so for a truth in the interior of the class with a degree $J_n \to \infty$.

Algorithm 1 runs L-BFGS-B in the boxes of the sieve from several starts and continues with an augmented Lagrangian when the best accepted run violates a class constraint (see `snapshot_control`). The selected candidate is feasible and passes the stopping rules, and its Karush-Kuhn-Tucker conditions are reported. Whether it attains the global supremum, as Theorem 5.6 assumes, is not checked; several starts reduce the risk of a local maximum.

Value

An object of class `snapshot_fit`, a list with components `success`, `status`, `model` (the fitted model), `par` (its working coordinates), `loglik`, `n`, `diagnostics` (constraint violation, stationarity of the Lagrangian, complementarity and multipliers), `constraint_details`, `box_runs`, `candidates`, `selected_run`, `elapsed` and `call`.

See Also

`snapshot_spec`, `snapshot_control`, `snapshot_covariance`, `snapshot_reconstruct`, `snapshot_study_data`

Examples

```
## Not run:
settings <- snapshot_study_settings()

# Study A: regular truth, n = 2000, first replication
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
fit

# relative covariance error on the evaluation grid
grid <- settings$grid
w <- outer(snapshot_trapezoid_weights(grid), snapshot_trapezoid_weights(grid))
C <- snapshot_covariance(d$truth, grid)
C_hat <- snapshot_covariance(fit, grid)
sqrt(sum(w * (C_hat - C)^2) / sum(w * C^2))

# Study D: three-point scores, restricted and atomic sieves
d <- snapshot_study_data("D", "atomic", 1000)
fit_normal <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
```

```
fit_atomic <- snapshot_fit(d$train, settings$atomic_spec, settings$atomic_control,
                           seed = d$seed)
c(normal = fit_normal$loglik, atomic = fit_atomic$loglik)

## End(Not run)
```

`snapshot_fpca` *Eigenvalues and eigenfunctions of the covariance operator*

Description

The nonzero eigenvalues and the eigenfunctions of the integral operator with kernel C.

Usage

```
snapshot_fpca(object, time = seq(0, 1, length.out = 101))
```

Arguments

`object`	a `snapshot_fit` object, or a model from `snapshot_model` or `snapshot_truth`.
`time`	times at which the eigenfunctions are evaluated.

Details

The operator has rank at most two. With the Gram matrix $B = (\langle v_k, v_l \rangle)_{k,l=1}^2$ of the loadings in $L^2[0,1]$, eigenvalues $\lambda_1 \geq \lambda_2$ and orthonormal eigenvectors e_j, the eigenvalues of the operator are λ_1, λ_2 and its orthonormal eigenfunctions are $\varphi_j = \lambda_j^{-1/2}(e_{j1}v_1 + e_{j2}v_2)$. B is computed by Gauss-Legendre quadrature, which is exact for polynomial loadings.

Value

A list with `time`, `eigenvalues`, `eigenfunctions` (a column for each eigenvalue), the Gram matrix `loading_gram`, and `repeated_eigenvalue`, which is `TRUE` when the two eigenvalues coincide and the eigenfunctions are not unique.

See Also

`snapshot_covariance`

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

# relative error of the two eigenvalues
pc <- snapshot_fpca(d$truth, grid)
pc_hat <- snapshot_fpca(fit, grid)
sqrt(sum((pc_hat$eigenvalues - pc$eigenvalues)^2) / sum(pc$eigenvalues^2))
matplot(grid, pc_hat$eigenfunctions, type = "l", lty = 1, ylab = "eigenfunction")

## End(Not run)
```

`snapshot_frequency_gram` *Frequency kernels and Gram matrices of the snapshot law*

Description

The Gram matrices of Section 3.4 and the coefficients xi recovered from them.

Usage

```
snapshot_frequency_gram(
  object,
  time,
  frequency_radius = NULL,
  nodes = 80,
  rank_tolerance = 1e-10
)
```

Arguments

`object`	a `snapshot_fit` object, or a model with a `spec` component.
`time`	times in [0, 1].
`frequency_radius`	radius r of the frequency interval.
`nodes`	number of Gauss-Legendre nodes.
`rank_tolerance`	relative threshold on the singular values for the numerical rank.

Details

With the cumulant generating functions of the scores, put $N(u,t) = \log \psi_{P_{\theta,t}}(u) - iu\kappa_1(t) + u^2\kappa_2(t)/2$, the conditional cumulant transform of Y given T = t for |u| <= r, and, with the Euler operator $E = u\,\partial/\partial u$, $N_1 = EN$, $N_2 = E^2N$, $N_3 = \partial N/\partial t$, $N_4 = E\,\partial N/\partial t$ and $N_5 = \partial^2 N/\partial t^2$. The Gram matrix $G_{ij}(t) = \langle N_i, N_j\rangle$, i, j <= 4, in $L^2[-r,r]$, and $b_i(t) = \langle N_i, N_5\rangle$ are determined by the snapshot law. G(t) is positive definite outside a finite set of times, and at such times with $v_1(t)v_2(t) \neq 0$, $\xi(t) = G(t)^{-1}b(t)$ gives the coefficients of Proposition 3.2 (Lemma 3.7). The smallest eigenvalue of G(t) controls the constants of the local stability bounds of Theorem 3.8.

The kernels are evaluated at the nodes of a Gauss-Legendre rule on [-r, r]. The default radius r lies in a disc on which the characteristic functions of all score laws of the class have no zeros. This disc is small, and the Gram matrices on it are close to singular in floating point. A larger `frequency_radius` can be used while the characteristic functions of the score laws of the model stay away from zero; the function stops otherwise. The model must carry its `snapshot_spec`, as fitted models do.

Value

A list with `time`, `gram` (a 4 x 4 x length(time) array), `xi`, `numerical_rank`, `smallest_eigenvalue`, `condition_number`, `imaginary_residual`, `frequency_radius` and `conservative_analytic_radius_satisfied`.

See Also

snapshot_ode_coefficients

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

time <- seq(0.05, 0.95, by = 0.1)
g <- snapshot_frequency_gram(fit, time, frequency_radius = 0.5)
g$numerical_rank
g$smallest_eigenvalue
max(abs(g$xi - snapshot_ode_coefficients(fit, time)$xi))

## End(Not run)
```

---

snapshot_loadings *Loadings and their derivatives*

Description

The loadings (v_1, v_2), or their first or second derivatives, of a fitted or a true model.

Usage

```
snapshot_loadings(object, time, derivative = 0)
```

Arguments

object	a snapshot_fit object, or a model from snapshot_model or snapshot_truth.
time	times in [0, 1].
derivative	order of the derivative: 0, 1 or 2.

Details

Under the conditions of Theorem 3.4, the snapshot law determines the loadings up to a signed permutation S of the two factors, $\tilde{v}(t) = Sv(t)$, and C does not depend on S. Errors of estimated loadings are therefore computed after alignment, as in the metric d of Theorem 5.4; see snapshot_align_model.

Value

A matrix with a row for each time and a column for each factor.

See Also

snapshot_align_model, snapshot_covariance

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

# sup-norm loading error after the signed permutation that matches the truth
aligned <- snapshot_align_model(fit, d$truth, grid)
max(abs(snapshot_loadings(aligned, grid) - snapshot_loadings(d$truth, grid)))
matplot(grid, snapshot_loadings(aligned, grid), type = "l", lty = 1, ylab = "v(t)")
matlines(grid, snapshot_loadings(d$truth, grid), lty = 2)

## End(Not run)
```

---

snapshot_mean *Mean function*

Description

The mean $\mu(t) = \sum_j \mu_j T_j(2t - 1)$ of a fitted or a true model.

Usage

```
snapshot_mean(object, time)
```

Arguments

`object`	a `snapshot_fit` object, or a model from `snapshot_model` or `snapshot_truth`.
`time`	times in [0, 1].

Details

The mean is identified from the first conditional cumulant, $\mu(t) = E(Y \mid T = t)$, for any score laws.

Value

A vector.

See Also

`snapshot_covariance`, `snapshot_moments`

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

# sup-norm error of the mean on the evaluation grid
```

```
max(abs(snapshot_mean(fit, grid) - snapshot_mean(d$truth, grid)))

## End(Not run)
```

`snapshot_model` *Model object*

Description

A model given by its mean coefficients, loading coefficients, noise variance and two score laws.

Usage

```
snapshot_model(mu_coef, v_coef, sigma2, score, spec = NULL)
```

Arguments

`mu_coef`	coefficients $\mu_0, \mu_1, \ldots$ of the mean.
`v_coef`	matrix of the coefficients of the loadings, a column for each factor.
`sigma2`	noise variance.
`score`	list of two score laws, from `snapshot_normal_score` or `snapshot_atomic_score`.
`spec`	optional `snapshot_spec` stored with the model.

Details

The mean is $\mu(t) = \sum_j \mu_j T_j(2t-1)$ and the loadings are $v_k(t) = \sum_j v_{kj} T_j(2t-1)$, with the Chebyshev polynomials T_j and no half factor on T_0. The score laws must be centred and standardized. Models are used as truths in `snapshot_simulate` and in all functions for fitted quantities.

Value

A list with `mu_coef`, `v_coef`, `sigma2`, `score`, `valid` and `spec`.

See Also

`snapshot_truth`, `snapshot_simulate`

Examples

```
## Not run:
# the regular truth of the simulation study, built by hand
score <- list(snapshot_normal_score(c(qlogis(0.3), 1.8, log(1.2))),
              snapshot_normal_score(c(qlogis(0.65), -1.5, log(0.85))))
model <- snapshot_model(c(0.3, 0.4, -0.15), cbind(c(1, 0.55, 0), c(0.15, 0.9, 0.15)),
                        0.08, score)
all.equal(model, snapshot_truth("regular"))

## End(Not run)
```

`snapshot_model_diagnostics` *Class constraints of a model*

Description

Values of the functionals that define the parameter class of Section 5.1, and the slack of each constraint.

Usage

```
snapshot_model_diagnostics(object, spec = NULL)
```

Arguments

`object`	a `snapshot_fit` object, or a model from `snapshot_model` or `snapshot_truth`.
`spec`	a `snapshot_spec`. By default the one stored in the model, or the default class for the degrees of the model.

Details

The class requires $\alpha(v) \geq \alpha_0$, $\gamma(v) \geq \gamma_0$, $\tau(v) \geq \tau_0$, $\mathrm{ND}(F_k) \geq \delta_0$, $E\, e^{c_0|\zeta_k|} \leq M_0$, the noise range and the coefficient boxes (see `snapshot_spec`). The functionals alpha and gamma are smallest eigenvalues of normalized Gram matrices, computed exactly by Gauss-Legendre quadrature; tau is attained at an end point or at a real root of v' or v + v". Every truth of the simulation study lies in the class with the default constants. Membership of a particular sieve is not checked.

Value

A list with `indices` (alpha, gamma, tau and their Gram matrices), `non_gaussianity`, `exponential_moment`, `class_constraints` (nonnegative when the constraint holds), `in_compact_class` and `score_moments`.

See Also

`snapshot_spec`, `snapshot_score_nd`

Examples

```
## Not run:
# functionals of the simulation truths, with the constants of the study
design <- snapshot_design()
cells <- design[!duplicated(design$truth), ]
settings <- snapshot_study_settings()
truth_of <- function(scenario, parameter) switch(scenario,
  delta = snapshot_truth("delta", delta = parameter),
  beta = snapshot_truth("beta", beta = parameter),
  snapshot_truth(scenario))
truths <- t(sapply(seq_len(nrow(cells)), function(i) {
  spec <- if (cells$scenario[i] == "atomic") settings$atomic_spec else settings$spec
  check <- snapshot_model_diagnostics(truth_of(cells$scenario[i], cells$parameter[i]), spec)
  c(alpha = check$indices$alpha, gamma = check$indices$gamma, tau = check$indices$tau,
    nd1 = check$non_gaussianity[1], nd2 = check$non_gaussianity[2],
    in_class = check$in_compact_class)
```

```
}))
rownames(truths) <- cells$truth
round(truths, 4)

## End(Not run)
```

`snapshot_moments` *Mean and variances of the latent process and the response*

Description

The mean, the variance $C(t, t)$ of X(t), and the variance $C(t, t) + \sigma^2$ of Y given T = t.

Usage

```
snapshot_moments(object, time)
```

Arguments

`object`	a `snapshot_fit` object, or a model from `snapshot_model` or `snapshot_truth`.
`time`	times in [0, 1].

Details

The second conditional cumulant $\kappa_2(t) = C(t, t) + \sigma^2$ is all that a Gaussian working likelihood can identify; non-Gaussian scores separate C(t, t) from σ^2 when 1, v_1^2 and v_2^2 are linearly independent.

Value

A data frame with columns `time`, `mean`, `latent_variance` and `response_variance`.

See Also

`snapshot_cumulant_curves`

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

m_hat <- snapshot_moments(fit, grid)
m <- snapshot_moments(d$truth, grid)
c(noise_error = abs(fit$model$sigma2 - d$truth$sigma2),
  variance_sup = max(abs(m_hat$latent_variance - m$latent_variance)))

## End(Not run)
```

`snapshot_normal_score` *Score law of the restricted sieve*

Description

The standardized two-component normal mixture F_ϑ of the restricted sieve.

Usage

```
snapshot_normal_score(theta)
```

Arguments

`theta`	the vector $(\vartheta_1, \vartheta_2, \vartheta_3)$.

Details

For $\vartheta = (\vartheta_1, \vartheta_2, \vartheta_3)$, let $w = 1/(1 + e^{-\vartheta_1})$, $m_1 = \{(1-w)/w\}^{1/2}\vartheta_2/(1+\vartheta_2^2)^{1/2}$, $m_2 = -\{w/(1-w)\}^{1/2}\vartheta_2/(1+\vartheta_2^2)^{1/2}$, $\varsigma_2^2 = 1/\{(1+\vartheta_2^2)(we^{2\vartheta_3} + 1 - w)\}$ and $\varsigma_1 = e^{\vartheta_3}\varsigma_2$. Then $F_\vartheta = wN(m_1, \varsigma_1^2) + (1-w)N(m_2, \varsigma_2^2)$ has mean zero and variance one for every ϑ. The restricted sieve takes ϑ in the box of `snapshot_spec`. The scores of the regular truth of the simulation study have $\vartheta = (\mathrm{logit}\, 0.3, 1.8, \log 1.2)$ and $(\mathrm{logit}\, 0.65, -1.5, \log 0.85)$.

Value

A score law: a list with `type`, `weight`, `mean`, `variance`, `theta` and `valid`.

See Also

`snapshot_atomic_score`, `snapshot_model`, `snapshot_score_nd`

Examples

```
## Not run:
# score laws of the regular and the separated truths (Study A)
regular <- snapshot_normal_score(c(qlogis(0.3), 1.8, log(1.2)))
separated <- snapshot_normal_score(c(qlogis(0.3), 4, log(1.2)))
x <- seq(-4, 4, by = 0.01)
plot(x, snapshot_score_density(separated, x), type = "l", ylab = "density")
lines(x, snapshot_score_density(regular, x), lty = 2)
c(regular = snapshot_score_nd(regular), separated = snapshot_score_nd(separated))

## End(Not run)
```

`snapshot_ode_coefficients` *Coefficients of the differential equation of the cumulant curves*

Description

The functions $\xi_1, \ldots, \xi_4$ of Proposition 3.2, computed from the loadings of a model.

Usage

```
snapshot_ode_coefficients(object, time, tolerance = 1e-08)
```

Arguments

`object`	a `snapshot_fit` object, or a model from `snapshot_model` or `snapshot_truth`.
`time`	times in [0, 1].
`tolerance`	threshold below which a loading or a difference of logarithmic derivatives is treated as zero.

Details

Let $\ell_k = v_k'/v_k$ and

$$\xi_1 = \frac{\ell_1\ell_2' - \ell_2\ell_1'}{\ell_1 - \ell_2}, \quad \xi_2 = -\ell_1\ell_2, \quad \xi_3 = \frac{\ell_1' - \ell_2'}{\ell_1 - \ell_2}, \quad \xi_4 = \ell_1 + \ell_2.$$

Where $v_1 v_2 W[v_1, v_2] \neq 0$, with W the Wronskian, every cumulant curve of order r >= 3 satisfies

$$\kappa_r'' = (\xi_3 + r\xi_4)\kappa_r' + (r\xi_1 + r^2\xi_2)\kappa_r,$$

with the same four functions for every r. The logarithmic derivatives are the roots of $z^2 - \xi_4 z - \xi_2$, so (ξ_2, ξ_4) determine them up to their labels, and hence each loading up to a constant factor. `snapshot_frequency_gram` recovers the same coefficients from the snapshot law.

Value

A list with `time`, `xi` (a matrix with `NA` where the coefficients are not defined), `logarithmic_derivative` and `valid`.

See Also

`snapshot_frequency_gram`, `snapshot_cumulant_curves`

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

time <- seq(0.05, 0.95, by = 0.1)
xi <- snapshot_ode_coefficients(d$truth, time)$xi
```

```
xi_hat <- snapshot_ode_coefficients(fit, time)$xi
round(cbind(xi, xi_hat), 3)

## End(Not run)
```

`snapshot_posterior_scores` *Conditional law of the scores given one snapshot*

Description

Conditional means and covariances of the scores (ζ_1, ζ_2) given one observation (T_i, Y_i) of each subject.

Usage

```
snapshot_posterior_scores(object, time, response)
```

Arguments

`object`	a `snapshot_fit` object, or a model from `snapshot_model` or `snapshot_truth`.
`time`, `response`	observation times and responses, one pair for each subject.

Details

Given the mixture component c = (j, l), the scores have independent normal laws with means a_c and variances D_c (point masses for the atomic sieve), and $Y = \mu(T) + v(T)^\top \zeta + \varepsilon$. Hence, given (T_i, Y_i) and c, the scores are normal with mean $a_c + D_c v(T_i)\{Y_i - m_c(T_i)\}/s_c^2(T_i)$ and covariance $D_c - D_c v(T_i) v(T_i)^\top D_c / s_c^2(T_i)$, where m_c and s_c^2 are the component mean and variance of Y given T, and the components have posterior probabilities proportional to $w_c \phi\{Y_i; m_c(T_i), s_c^2(T_i)\}$. The parameters of the model are plugged in.

Value

A list with `mean` (a matrix with two columns), `covariance` (a 2 x 2 x n array) and `mixture_probability` (a row for each subject and a column for each mixture component).

See Also

`snapshot_reconstruct`, `snapshot_align_model`

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "separated", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)

# predicted against true scores of the test subjects: fitted model after alignment,
# and the oracle with the true parameters
aligned <- snapshot_align_model(fit, d$truth, settings$grid)
test <- d$test$data
fitted <- snapshot_posterior_scores(aligned, test$time, test$response)
```

```
oracle <- snapshot_posterior_scores(d$truth, test$time, test$response)
rbind(fitted = diag(cor(fitted$mean, d$test$scores)),
      oracle = diag(cor(oracle$mean, d$test$scores)))

## End(Not run)
```

---

snapshot_quantile *Conditional quantiles of the response given the time*

Description

Quantiles of Y given T = t under a fitted or a true model.

Usage

```
snapshot_quantile(object, time, probability = c(0.025, 0.5, 0.975))
```

Arguments

object	a snapshot_fit object, or a model from snapshot_model or snapshot_truth.
time	times in [0, 1].
probability	probabilities in [0, 1].

Details

The quantile of order p is the root of $F(y \mid t) = p$ for the normal mixture of snapshot_cdf, computed by uniroot. Pairs of quantiles give prediction intervals for a new snapshot at time t.

Value

A matrix with a row for each time and a column for each probability.

See Also

snapshot_cdf, snapshot_reconstruct_quantile

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

# coverage of the 90 percent prediction intervals for the test snapshots
test <- d$test$data
q <- snapshot_quantile(fit, test$time, c(0.05, 0.95))
mean(test$response >= q[, 1] & test$response <= q[, 2])

# quantile curves of Y given T, fitted and true
matplot(grid, snapshot_quantile(fit, grid, c(0.05, 0.5, 0.95)), type = "l", lty = 1,
        col = 1, ylab = "y")
matlines(grid, snapshot_quantile(d$truth, grid, c(0.05, 0.5, 0.95)), lty = 2, col = 2)

## End(Not run)
```

`snapshot_reconstruct` *Prediction of a curve from one snapshot*

Description

Conditional mean and variance of the latent curve X(s), on a grid, given one observation (T_i, Y_i).

Usage

```
snapshot_reconstruct(
  object,
  time,
  response,
  grid = seq(0, 1, length.out = 101),
  variance = TRUE
)
```

Arguments

`object`	a `snapshot_fit` object, or a model from `snapshot_model` or `snapshot_truth`.
`time`, `response`	observation times and responses, one pair for each subject.
`grid`	times at which the curves are predicted.
`variance`	also return the conditional variances.

Details

$E\{X(s) \mid T_i, Y_i\} = \mu(s) + v(s)^\top E(\zeta \mid T_i, Y_i)$ and $\mathrm{var}\{X(s) \mid T_i, Y_i\} = v(s)^\top \mathrm{cov}(\zeta \mid T_i, Y_i) v(s)$, with the conditional law of the scores of `snapshot_posterior_scores`. The predictor is nonlinear in Y_i. The simulation study reports the mean integrated squared error of this prediction for new subjects, and the oracle error obtained with the true parameters.

Value

A list with `grid`, `mean` (a row for each subject and a column for each grid point), `variance` (the same layout, or `NULL`) and `posterior_scores`.

See Also

`snapshot_posterior_scores`, `snapshot_reconstruct_quantile`

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

# curve MISE for the 2000 test subjects: fitted model and oracle
test <- d$test$data
w <- snapshot_trapezoid_weights(grid)
ise <- function(pred) apply((pred$mean - d$test$curves)^2, 1, function(e) sum(w * e))
```

```
c(fitted = mean(ise(snapshot_reconstruct(fit, test$time, test$response, grid))),
  oracle = mean(ise(snapshot_reconstruct(d$truth, test$time, test$response, grid))))

## End(Not run)
```

`snapshot_reconstruct_quantile` *Pointwise prediction bands for a curve from one snapshot*

Description

Pointwise conditional quantiles of the latent curve X(s), on a grid, given one observation (T_i, Y_i).

Usage

```
snapshot_reconstruct_quantile(
  object,
  time,
  response,
  grid = seq(0, 1, length.out = 101),
  probability = c(0.05, 0.5, 0.95)
)
```

Arguments

`object`	a `snapshot_fit` object, or a model from `snapshot_model` or `snapshot_truth`.
`time`, `response`	observation times and responses, one pair for each subject.
`grid`	times at which the quantiles are computed.
`probability`	probabilities.

Details

Given (T_i, Y_i), X(s) is a finite mixture over the mixture components of the scores, with the posterior component probabilities as weights; its components are normal, or point masses for the atomic sieve (see `snapshot_posterior_scores`). The quantile of order p is the smallest x with $F(x) \geq p$, found by bisection, which is valid for continuous, discrete and mixed laws. The parameters of the model are plugged in, so the bands do not include estimation error.

Value

A list with `grid`, `quantile` (an array indexed by subject, grid point and probability) and `probability`.

See Also

`snapshot_reconstruct`

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "regular", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
grid <- settings$grid

# 90 percent pointwise bands for three test subjects
test <- d$test$data
band <- snapshot_reconstruct_quantile(fit, test$time[1:3], test$response[1:3], grid)
matplot(grid, t(band$quantile[, , "p0.5"]), type = "l", lty = 1, ylab = "X(s)")
for (i in 1:3) {
  lines(grid, band$quantile[i, , "p0.05"], lty = 2, col = i)
  lines(grid, band$quantile[i, , "p0.95"], lty = 2, col = i)
  lines(grid, d$test$curves[i, ], lty = 3, col = i)
  points(test$time[i], test$response[i], pch = 19, col = i)
}

## End(Not run)
```

---

snapshot_score_cdf *Distribution function of a score law*

Description

Distribution function of a score law of either sieve.

Usage

```
snapshot_score_cdf(score, x)
```

Arguments

score	a score law.
x	points at which the distribution function is evaluated.

Details

The distribution function is $\sum_j w_j \Phi\{(x - m_j)/s_j\}$, with a step at each atom of zero variance. The simulation study compares fitted and true score laws, after the alignment of `snapshot_align_model`, by the Kolmogorov distance.

Value

A vector of probabilities.

See Also

`snapshot_align_model`, `snapshot_score_density`

Examples

```
## Not run:
# the atomic truth of Study D against its fit on the atomic sieve
settings <- snapshot_study_settings()
d <- snapshot_study_data("D", "atomic", 1000)
fit <- snapshot_fit(d$train, settings$atomic_spec, settings$atomic_control, seed = d$seed)
aligned <- snapshot_align_model(fit, d$truth, settings$grid)
x <- sort(c(seq(-4, 4, length.out = 2001), aligned$score[[1]]$mean, d$truth$score[[1]]$mean))
plot(x, snapshot_score_cdf(d$truth$score[[1]], x), type = "s", ylab = "F(x)")
lines(x, snapshot_score_cdf(aligned$score[[1]], x), type = "s", col = 2)

## End(Not run)
```

---

snapshot_score_cumulants

Moments and cumulants of a score law

Description

Raw moments and cumulants of a score law up to a given order.

Usage

```
snapshot_score_cumulants(score, max_order = 8)
```

Arguments

score	a score law.
max_order	highest order, between 2 and 20.

Details

The moments of each normal component follow from $EZ^r = mEZ^{r-1} + (r-1)s^2EZ^{r-2}$ for $Z \sim N(m, s^2)$, and the cumulants from the moments by the recursion $\kappa_r = \mu_r - \sum_{k=1}^{r-1} \binom{r-1}{k-1} \kappa_k \mu_{r-k}$. The cumulants of order r >= 3 of the scores scale the cumulant curves of Y given T; see snapshot_cumulant_curves.

Value

A list with moment (orders 0 to max_order) and cumulant (orders 1 to max_order).

See Also

snapshot_cumulant_curves

Examples

```
## Not run:
# skewness of the score laws of the skewed truth (Study D)
sapply(snapshot_truth("skewed")$score, function(s) snapshot_score_cumulants(s, 3)$cumulant[3])

## End(Not run)
```

`snapshot_score_density` *Density of a normal-mixture score law*

Description

Density of a score law of the restricted sieve, or of another finite normal mixture.

Usage

```
snapshot_score_density(score, x)
```

Arguments

`score`	a score law with positive component variances.
`x`	points at which the density is evaluated.

Details

The density is $\sum_j w_j \phi(x; m_j, s_j^2)$. Atomic laws have no density; use `snapshot_score_cdf` for them.

Value

A vector.

See Also

`snapshot_score_cdf`, `snapshot_normal_score`

Examples

```
## Not run:
# score laws of the Study B truths, delta = 0.9 and 0.35
x <- seq(-4, 4, by = 0.01)
strong <- snapshot_truth("delta", delta = 0.9)$score[[1]]
weak <- snapshot_truth("delta", delta = 0.35)$score[[1]]
plot(x, snapshot_score_density(strong, x), type = "l", ylab = "density")
lines(x, snapshot_score_density(weak, x), lty = 2)
lines(x, dnorm(x), col = 2)

## End(Not run)
```

`snapshot_score_nd` *Non-Gaussianity of a score law*

Description

The index $\mathrm{ND}(F) = \max_{|u|\le 1} |\log \psi_F(u) + u^2/2|$, with ψ_F the characteristic function, evaluated on a grid of u.

Usage

```
snapshot_score_nd(score, u = seq(-1, 1, length.out = 401))
```

Arguments

`score`	a score law, from `snapshot_normal_score` or `snapshot_atomic_score`.
`u`	points at which the maximum is taken.

Details

For a law with mean zero and variance one, $|\psi_F(u) - 1| \le 1/2$ on [-1, 1], so the principal logarithm is defined, and for laws with an exponential moment ND(F) = 0 only for N(0, 1). The class of Section 5.1 requires $\mathrm{ND}(F_k) \ge \delta_0$. The maximum over the 401 points $-1 + j/200$ does not exceed ND(F), so a computed value at least δ_0 implies the exact constraint. For the scores of Study B, ND is of order δ^3, the regime of the lower bound of Theorem 4.1.

Value

A number.

See Also

`snapshot_model_diagnostics`, `snapshot_truth`

Examples

```
## Not run:
# ND of the two score laws of the Study B truths
sapply(c(0.9, 0.7, 0.5, 0.35), function(delta)
  sapply(snapshot_truth("delta", delta = delta)$score, snapshot_score_nd))

## End(Not run)
```

`snapshot_simulate` *Simulation of snapshot data*

Description

Draws snapshots, or repeated observations, from a model.

Usage

```
snapshot_simulate(
  n,
  model = NULL,
  scenario = c("regular", "separated", "near_gaussian", "gaussian", "atomic", "skewed",
    "delta", "beta"),
  visits = 1,
  grid = seq(0, 1, length.out = 101),
  seed = NULL,
  ...
)
```

Arguments

`n`	number of subjects.
`model`	a model or a `snapshot_fit` object. If `NULL`, the model `snapshot_truth(scenario, ...)`.
`scenario`	a scenario of `snapshot_truth`.
`visits`	number of observations of each curve.
`grid`	times at which the latent curves are returned.
`seed`	optional seed. The random number state of the session is restored afterwards.
`...`	`delta`, `beta` or `atoms`, passed to `snapshot_truth`.

Details

For each of n subjects, the scores are drawn from the score laws, the latent curve is X(t) = mu(t) + v_1(t) zeta_1 + v_2(t) zeta_2, the times are uniform on [0, 1], and Y = X(T) + eps with $\varepsilon \sim N(0, \sigma^2)$. With `visits` = 1 each subject gives one snapshot, the design of the paper. With `visits` = 2 each curve is observed at two independent uniform times; Study E gives this paired sample of n/2 subjects to the methods that need repeated observations, so that all methods use n measurements. The draws are made in a fixed order: the mixture component and score of factor 1, then of factor 2, the times subject by subject, and the noise.

Value

A list with `data` (from `snapshot_data` when `visits` = 1, otherwise a data frame with columns `id`, `visit`, `time` and `response`), `latent_at_time`, `scores`, `curves` (the latent curves on the grid, a row for each subject), `grid`, `truth`, `visits` and `scenario`.

See Also

`snapshot_study_data`, `snapshot_truth`

Examples

```
## Not run:
# snapshots from the Study B truth with delta = 0.5
sim <- snapshot_simulate(2000, scenario = "delta", delta = 0.5, seed = 1)
plot(sim$data$time, sim$data$response, pch = 20, cex = 0.5, xlab = "t", ylab = "y")
matlines(sim$grid, t(sim$curves[1:10, ]), lty = 1)

# the paired design of Study E: 1000 subjects observed twice
paired <- snapshot_simulate(1000, scenario = "regular", visits = 2, seed = 2)$data
head(paired)

## End(Not run)
```

`snapshot_spec` *Parameter class and sieve*

Description

Degrees of the Chebyshev expansions, the sieve for the score laws, and the constants of the compact parameter class of Section 5.1.

Usage

```
snapshot_spec(
  degree = 2,
  mean_degree = degree,
  sieve = c("normal", "atomic"),
  A = 3,
  rho = 1.5,
  A_mu = 2.5,
  rho_mu = 1.5,
  sigma2_min = 0.01,
  sigma2_max = 3,
  alpha_min = 0.01,
  gamma_min = 1e-04,
  tau_min = 0.02,
  nd_min = 0.002,
  c0 = 0.3,
  M0 = 10,
  w_min = 0.1,
  b_max = 6,
  r_max = 3,
  nd_points = 401
)
```

Arguments

`degree`	degree J of the loadings.
`mean_degree`	degree J_μ of the mean.
`sieve`	`"normal"` for the restricted sieve, `"atomic"` for the atomic sieve. The atomic sieve requires `mean_degree = degree`.

`A`, `rho` bounds on the loading coefficients.

`A_mu`, `rho_mu` bounds on the mean coefficients.

`sigma2_min`, `sigma2_max` range of the noise variance.

`alpha_min`, `gamma_min`, `tau_min` the constants α_0, γ_0 and τ_0.

`nd_min` the constant δ_0.

`c0`, `M0` the constants of the moment condition.

`w_min`, `b_max`, `r_max` box of the normal-mixture parameters: mixture weight in [w_min, 1 - w_min], separation |b| <= b_max, and ratio of the component standard deviations in [1/r_max, r_max].

`nd_points` number of equally spaced points of [-1, 1] at which ND is evaluated. With 401 points the computed ND does not exceed ND.

Details

The mean and the loadings are expansions in $T_j(2t-1)$, with $|\mu_j| \le A_\mu \rho_\mu^{-j}$ and $|v_{kj}| \le A\rho^{-j}$. With the normalized Gram matrix Γ, the class requires $\alpha(v) = \lambda_{\min}\{\Gamma(v_1, v_2)\} \ge \alpha_0$, $\gamma(v) = \lambda_{\min}\{\Gamma(1, v_1^2, v_2^2)\} \ge \gamma_0$ and $\tau(v) = \min_k \min_t \{v_k(t)^2 + v_k'(t)^2\}^{1/2} \ge \tau_0$, score laws with mean zero, variance one, $E\, e^{c_0|\zeta_k|} \le M_0$ and $\mathrm{ND}(F_k) \ge \delta_0$, where $\mathrm{ND}(F) = \max_{|u|\le 1} |\log \psi_F(u) + u^2/2|$ and ψ_F is the characteristic function, and $\sigma^2_{\min} \le \sigma^2 \le \sigma^2_{\max}$. These bounds separate the class from linearly dependent loadings, variance confounding and Gaussian scores. The constants are fixed before the data are examined; membership of the true parameter in the class is an identifying assumption.

The restricted sieve Θ_J^{N} has a mean of degree `mean_degree`, loadings of degree J = `degree`, and score laws F_ϑ (see `snapshot_normal_score`) with ϑ in the box given by `w_min`, `b_max` and `r_max`. The atomic sieve Θ_J^{A} has mean and loadings of degree J and standardized score laws on the grid of the points j/J, |j| <= J^2, with grid variance at least 1/2 (see `snapshot_atomic_score`). For the restricted sieve the moment condition holds on the whole box when `M0` is at least $2\exp(c_0\bar{m} + c_0^2\bar{\varsigma}^2/2)$, the bound returned as `normal_moment_bound`; it is then not imposed as a constraint. The defaults are the constants of the simulation study.

Value

A list of class `snapshot_spec` with the arguments and `mixture_box`, `grid` (atomic sieve), `normal_moment_bound` and `moment_constraints`.

References

Beyaztas, U., Norouzirad, M. and Bandyopadhyay, S. (2026). The covariance of a random function from one observation per curve. Manuscript.

See Also

`snapshot_control`, `snapshot_fit`, `snapshot_model_diagnostics`

Examples

```
## Not run:
# the two sieves of the simulation study, J = J_mu = 2
spec <- snapshot_spec(degree = 2, mean_degree = 2)
spec$normal_moment_bound
spec$moment_constraints
atomic_spec <- snapshot_spec(degree = 2, mean_degree = 2, sieve = "atomic")
atomic_spec$grid

## End(Not run)
```

`snapshot_study_data` *Samples of one replication of the simulation study*

Description

The training sample, the test sample and, in Study E, the paired sample of one replication of a cell, drawn with the seeds of the simulation study.

Usage

```
snapshot_study_data(
  study,
  scenario,
  n,
  parameter = NA,
  replication = 1,
  n_test = 2000,
  grid = seq(0, 1, length.out = 101)
)
```

Arguments

`study`	letter of the study, from `"A"` to `"E"`.
`scenario`	truth of the cell, a scenario of `snapshot_truth`.
`n`	number of training snapshots.
`parameter`	delta for `"delta"` and beta for `"beta"`; not used otherwise.
`replication`	index of the replication.
`n_test`	number of test snapshots.
`grid`	times at which the latent curves are returned.

Details

The simulation study has five studies, each cell with 100 replications. Study A: truths `"regular"` and `"separated"`, n = 500, 1000, 2000, 5000, 10000 and 20000. Study B: truth `"delta"` with delta = 0.9, 0.7, 0.5 and 0.35, n = 2000 and 10000. Study C: truth `"beta"` with beta = 8, 2, 0.5 and 0.1, n = 2000 and 10000. Study D: truths `"atomic"` and `"skewed"`, n = 1000, 2000 and 5000, fitted on both sieves. Study E: truth `"regular"`, n = 1000, 2000 and 5000, with a paired sample of n/2 subjects observed twice for the methods that need repeated observations.

Replication r of the cell of study s (A = 1, ..., E = 5) with n training snapshots uses the seed 1e8 s + 1e3 n + r for the training sample and for the fit, this seed plus 5e8 for the test sample, and this seed plus 1e9 for the paired sample of Study E. These are the draws of the simulation study.

Value

A list with `train` (the training data), `sample` and `test` (outputs of `snapshot_simulate`), `pairs` (Study E; otherwise `NULL`), `truth`, `seed`, and the description of the cell.

See Also

`snapshot_design`, `snapshot_study_settings`

Examples

```
## Not run:
settings <- snapshot_study_settings()

# one replication of each study
dA <- snapshot_study_data("A", "regular", 500)
dB <- snapshot_study_data("B", "delta", 10000, parameter = 0.35)
dC <- snapshot_study_data("C", "beta", 2000, parameter = 0.1)
dD <- snapshot_study_data("D", "skewed", 1000)
dE <- snapshot_study_data("E", "regular", 2000)
head(dE$pairs)

fit <- snapshot_fit(dC$train, settings$spec, settings$control, seed = dC$seed)
c(sigma2 = fit$model$sigma2, true = dC$truth$sigma2)

## End(Not run)
```

---

snapshot_study_settings

Settings of the proposed estimator in the simulation study

Description

Sieves, numerical options, evaluation grid and test sample size of the simulation study.

Usage

```
snapshot_study_settings(trace = FALSE)
```

Arguments

`trace`	passed to `snapshot_control`.

Details

The restricted sieve Θ_2^{N} with J = J_mu = 2 is fitted by Algorithm 1 with twelve starting values and the analytic gradient. In Study D the atomic sieve Θ_2^{A} is also fitted, with four starting values, at most 5000 iterations per run and 200 EM iterations that refine each start. Errors are evaluated on 101 equally spaced points of [0, 1], and predictions on 2000 test snapshots. The class constants are the defaults of `snapshot_spec`.

Value

A list with `spec`, `control`, `atomic_spec`, `atomic_control`, `grid` and `n_test`.

See Also

snapshot_study_data, snapshot_fit

Examples

```
## Not run:
settings <- snapshot_study_settings()
d <- snapshot_study_data("A", "separated", 2000)
fit <- snapshot_fit(d$train, settings$spec, settings$control, seed = d$seed)
fit

## End(Not run)
```

---

snapshot_trapezoid_weights

Trapezoid weights

Description

Weights of the trapezoid rule on an increasing grid.

Usage

```
snapshot_trapezoid_weights(grid)
```

Arguments

grid	increasing grid.

Details

With these weights w, $\sum_j w_j f(s_j)$ approximates $\int f$, and `outer(w, w)` gives the double integrals of the L2 errors of covariance and correlation surfaces used in the simulation study.

Value

A vector of weights.

See Also

snapshot_covariance, snapshot_reconstruct

Examples

```
## Not run:
grid <- seq(0, 1, length.out = 101)
w <- snapshot_trapezoid_weights(grid)
sum(w * grid^2)

## End(Not run)
```

`snapshot_truth` *True models of the simulation study*

Description

The models of the simulation study of Section 6.

Usage

```
snapshot_truth(
  scenario = c("regular", "separated", "near_gaussian", "gaussian", "atomic", "skewed",
    "delta", "beta"),
  delta = 0.7,
  beta = 1,
  atoms = 50
)
```

Arguments

`scenario`	name of the model.
`delta`	weight of the two-point part in `"delta"`, in (0, 0.99].
`beta`	parameter of `"beta"`, nonnegative.
`atoms`	number of atoms of each score law in `"skewed"`, at least 10.

Details

All models have the mean $\mu(t) = 0.3 + 0.4\,T_1(x) - 0.15\,T_2(x)$, x = 2t - 1, and the noise variance 0.08. All except `"beta"` have the loadings $v_1 = 1 + 0.55\,T_1$ and $v_2 = 0.15 + 0.9\,T_1 + 0.15\,T_2$.

regular, separated Two-component normal mixture scores in the restricted sieve; the scores of `"separated"` are more bimodal. Study A, and `"regular"` in Study E.

delta Scores $(1 - \delta^2)^{1/2} G_k + \delta Z_k$, with G_k standard normal and standardized two-point Z_k, $P(Z_1 > 0) = 0.3$ and $P(Z_2 < 0) = 0.65$: the construction of Section 4.1. The model lies in the restricted sieve, and ND is of order δ^3. Study B, with delta = 0.9, 0.7, 0.5 and 0.35.

beta Loadings the degree-2 Chebyshev truncation of $\{(1 + \beta t)/(1 + \beta/2)\}^{1/2}(\cos, \sin)(\pi/6 + \pi t/2)$, so that $v_1^2 + v_2^2$ is close to $(1 + \beta t)/(1 + \beta/2)$ and gamma(v) is small for small beta, the mechanism of Proposition 4.2. Scores and noise of `"regular"`. Study C, with beta = 8, 2, 0.5 and 0.1.

atomic Standardized three-point scores, in the atomic sieve for J >= 2 and outside the restricted sieve. Study D.

skewed Equal-probability discretizations, with `atoms` points, of a standardized Gamma(4) law and of minus a standardized Gamma(9) law, with skewness 0.92 and -0.62 for 50 atoms; outside both sieves. Study D.

near_gaussian, gaussian Scores with ND below the default `nd_min`, and normal scores, for which C is not identified. They are not part of the simulation study.

All models of the simulation study lie in the parameter class with the default constants of `snapshot_spec`.

Value

A model; see snapshot_model.

References

Beyaztas, U., Norouzirad, M. and Bandyopadhyay, S. (2026). The covariance of a random function from one observation per curve. Manuscript.

See Also

snapshot_simulate, snapshot_design, snapshot_model_diagnostics

Examples

```
## Not run:
grid <- seq(0, 1, length.out = 101)

# Study C: the index gamma(v) of the four truths
sapply(c(8, 2, 0.5, 0.1), function(beta)
  snapshot_model_diagnostics(snapshot_truth("beta", beta = beta))$indices$gamma)

# covariance of the regular truth
persp(grid, grid, snapshot_covariance(snapshot_truth("regular"), grid),
      theta = 30, phi = 30, zlab = "C(s, t)")

## End(Not run)
```

Index

optim, *8*

print.snapshot_fit *(*snapshot_fit*)*, 17

snapshot_align_model, 3, *21*, *28*, *32*
snapshot_atomic_score, 4, *23*, *26*, *35*, *38*
snapshot_bin_cumulants, *3*, 5, *11*, *15, 16*
snapshot_cdf, *3*, 6, *13*, *29*
snapshot_control, *3*, 7, *17, 18*, *38*, *40*
snapshot_correlation, *3*, 9, *10*
snapshot_covariance, *3*, *9*, 10, *18, 19*, *21, 22*, *41*
snapshot_cumulant_curves, *3*, *5*, 11, *16*, *25*, *27*, *33*
snapshot_data, 12, *17*, *36*
snapshot_density, *3*, *6*, 13
snapshot_design, *3*, 14, *40*, *43*
snapshot_empirical_cumulants, *3*, *5, 6*, 15
snapshot_encode, 16, *17*
snapshot_fit, *3*, *7, 8*, *12*, *16*, 17, *38*, *41*
snapshot_fpca, *3*, *10*, 19
snapshot_frequency_gram, *3*, 20, *27*
snapshot_loadings, *3, 4*, 21
snapshot_mean, *3*, 22
snapshot_model, *3*, *5, 6*, *9–11*, *13*, *19*, *21, 22*, 23, *24–31*, *43*
snapshot_model_diagnostics, *3*, 24, *35*, *38*, *43*
snapshot_moments, *3*, *22*, 25
snapshot_normal_score, *5*, *23*, 26, *34, 35*, *38*
snapshot_ode_coefficients, *3*, *11*, *21*, 27
snapshot_posterior_scores, *3*, 28, *30, 31*
snapshot_quantile, *3*, *6*, *13*, 29
snapshot_reconstruct, *3*, *18*, *28*, 30, *31*, *41*
snapshot_reconstruct_quantile, *3*, *29, 30*, 31
snapshot_score_cdf, *4*, 32, *34*
snapshot_score_cumulants, 33
snapshot_score_density, *32*, 34
snapshot_score_nd, *24*, *26*, 35
snapshot_simulate, *3*, *12*, *23*, 36, *40*, *43*
snapshot_spec, *3*, *5*, *16–18*, *24*, *26*, 37, *40*, *42*
snapshot_study_data, *3*, *14*, *18*, *36*, 39, *41*
snapshot_study_settings, *3*, *8*, *14*, *40*, 40
snapshot_trapezoid_weights, *10*, 41
snapshot_truth, *3*, *5, 6*, *9–11*, *13*, *19*, *21–25*, *27–31*, *35, 36*, *39*, 42
SnapshotFDA *(*SnapshotFDA-package*)*, 2
SnapshotFDA-package, 2

uniroot, *29*